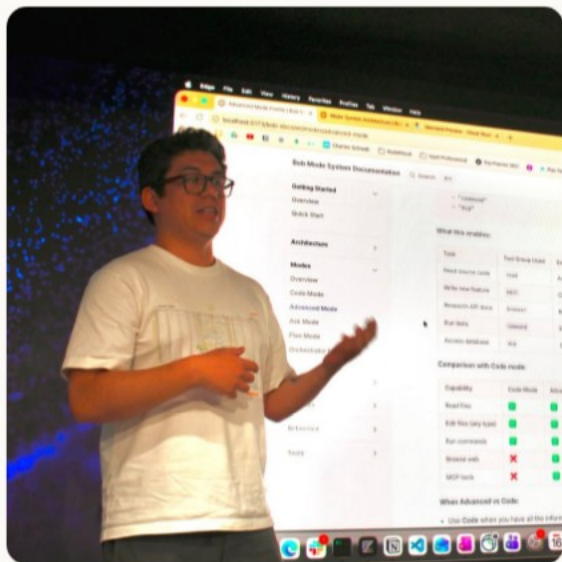


Tu agente de IA es **stateless**: lo que DevOps ya sabía sobre la memoria

Compactas el contexto y se pierde todo. No es un bug del modelo: es un problema de sistemas que DevOps lleva una década resolviendo.

NOMBRE DE SPEAKER **Rodolfo Castelo** • HASHICORP / IBM





Rodolfo Castelo Méndez

SOLUTIONS ARCHITECT SPECIALIST • HASHICORP / IBM

¡Hola! Soy Rodolfo Castelo. Llevo cerca de 10 años en tecnología, aunque vengo del mundo de las redes y comunicaciones, un ámbito bastante distinto. Hoy soy Solutions Architect Specialist en HashiCorp / IBM, donde trabajo con Terraform, Vault y HCP, y desde ahí me metí de lleno en workflows de IA, construyendo sistemas multiagente, el mismo terreno del que vengo a hablarles hoy. Antes pasé por consultoras, lo que me permitió tocar de todo: desde soporte helpdesk hasta migraciones de infraestructura de secretos, despliegues de infraestructura como código y migraciones de OpenShift a GKE en grandes empresas, de banca a telecom. Esa diversidad de proyectos y tecnologías es algo que hoy agradezco. Si algo me caracteriza es optimizar el tiempo: lo que se puede automatizar, lo automatizo.

Qué vamos a ver hoy.

01

Un PR, una pregunta.

02

El contexto que se pierde

03

Dónde vive la memoria

04

Cuatro patrones que ya
conoces

05

Lo que se rompe

06

Qué hacer mañana

Un pull request real. Dos rondas de auditoría automatizada (seguridad, patrones, cobertura de tests) antes de llegar a main.

YUI-SOUL

PLAN 192

Crew de agentes de IA · proyecto propio, uso diario

Race condition en lock distribuido: 1 crítico + 2 altos sobre suite verde (224/224) → 0 hallazgos



¿Quién escribió ese código?



Un agente.

No un colaborador.

75% del código nuevo en Google ya es generado por IA. Sundar Pichai, Google Cloud Next '26, abril 2026.

STATELESS

No retiene nada entre sesiones. Si necesita estado, lo busca afuera, exactamente como cualquier servicio stateless que ya han puesto en producción.

[sesión anterior](#)[decisiones tomadas](#)[reglas no escritas](#)[...](#)

Tres semanas de contexto. Cero memoria un viernes.

01

Librería descartada

El martes decidieron no usarla. El viernes, el agente vuelve a proponerla.

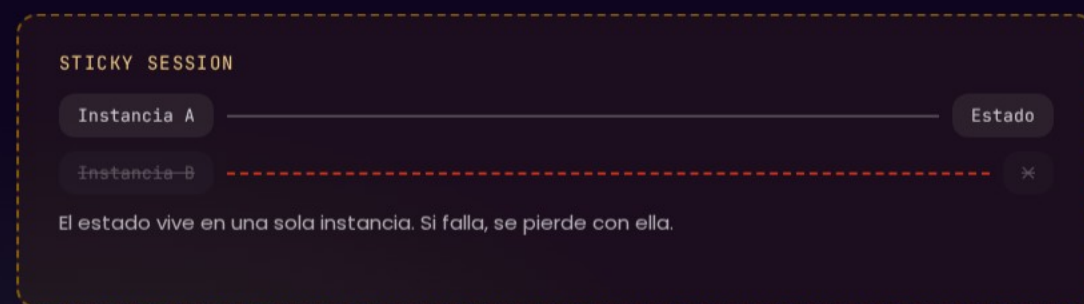
02

Bug re-diagnosticado

El miércoles encontraron la causa raíz. El viernes, empieza desde cero.

Esto ya lo conocen: session affinity.

"Ventana de contexto más grande" es la misma apuesta que sticky sessions: compra tiempo, no resuelve nada.



¿Dónde tiene que vivir el estado para sobrevivir a la próxima sesión?

Ya la resolvieron para bases de datos, colas, logs distribuidos. Todavía no para lo que un agente de IA sabe sobre su proyecto.

El estado vive afuera.

Filesystem + git.

¿Por qué estos dos lugares?

GIT

Versión, auditoría, diff

Cada decisión, si vive en un archivo commiteado, tiene autor, timestamp y un mensaje que explica el porqué. Historial gratis.

FS

Estructura, grep, durabilidad

Un plan, una spec, un archivo de decisiones: texto plano, ubicable, buscable. Sobrevive a que cierren la sesión.

ESTO ES GITOPS APLICADO A AGENTES

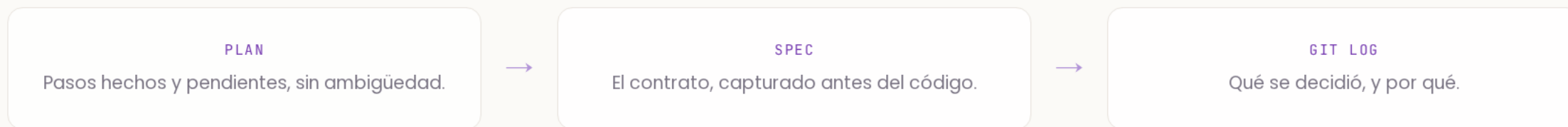
Estado versionado, auditable, fuera del componente, reconstruible desde cero.



La sesión nueva no recuerda. Lee.

Un agente pierde todo su contexto a mitad de tarea. La siguiente sesión (u otro agente distinto) no necesita que nadie le "recuerde" nada:

 SESIÓN REINICIADA: CONTEXTO PERDIDO -----



Continúa exactamente donde se quedó. Sin improvisar.

01

RFC antes de planificar.

"¿Por qué esta arquitectura y no otra?" se responde antes, por escrito, con espacio para que alguien la cuestione, antes de que exista un plan.

PARALELO DEVOPS → ADR, DESIGN DOC, RFC DE TODA LA VIDA

02

Plan-first, siempre.

Ningún agente escribe código sin que exista antes un plan versionado en el filesystem, con pasos explícitos y criterios de "done".

PARALELO DEVOPS → PLAN DE CAMBIO, RUNBOOK, TICKET ANTES DE TOCAR PRODUCCIÓN

03

TDD al nivel del workflow.

Especificación, test, implementación y revisión se encadenan: cada etapa exige el visto bueno de la anterior antes de avanzar. No hay atajos entre pasos.

PARALELO DEVOPS → SHIFT-LEFT TESTING, CONTRACT-FIRST DEVELOPMENT

TDD no desaparece: sigue viviendo dentro de "implementación". SDD es el paraguas spec-first que lo contiene.

04

Lessons-learned continuo.

Cada tarea que termina, falle o no, captura qué se aprendió, en disco, versionado e indexado. La próxima sesión lo encuentra sin redescubrirlo.

PARALELO DEVOPS → POSTMORTEM SIN CULPA, ACTUALIZAR EL RUNBOOK

Ninguna receta es perfecta.

EL PLAN SE VUELVE MENTIRA

Nadie lo actualiza. El agente lo trata como la verdad actual, porque es la única que tiene.

SOBRE-APLICAR EL PROCESO

Ceremonia completa para un cambio de una línea. Técnicamente correcto, prácticamente teatro.

EL CONOCIMIENTO SE VUELVE RUIDO

Sin curación, las lessons-learned se acumulan y nadie las vuelve a leer. Alert fatigue, aplicado a conocimiento.

Tres cosas que puedes hacer mañana.

1

MAÑANA EN LA MAÑANA

Abre el plan del proyecto y pregúntate si todavía dice la verdad.

↳ Responde a: El plan se vuelve mentira

2

ESTA SEMANA

Decide en voz alta el umbral que separa un cambio que necesita RFC de uno que no.

↳ Responde a: Sobre-aplicar el proceso

3

LA PRÓXIMA VEZ

Cuando una tarea con un agente termine, escribe tres líneas de lo aprendido, en disco.

↳ Responde a: El conocimiento se vuelve ruido

El mismo principio, otro backend.

El principio no es "usa git": es sacar el estado del componente y hacerlo versionado, auditable y durable. Git y el filesystem son lo que ustedes ya conocen y en lo que ya confían.



Ejemplo ilustrativo: un índice SQLite derivado de archivos versionados en git, reconstruible en cualquier momento, sin ruta de escritura propia. No reemplaza el estado, lo hace más rápido de consultar.

¿Quién escribió ese código?



Ya no importa quién escribió el código.

Importa dónde vive el conocimiento.

Eso es lo que DevOps ya sabía sobre la memoria.

yui-soul, por dentro.

Yui orquesta: decide qué hace falta, delega en el agente correcto, revisa el resultado y mantiene el trabajo coherente. Cada miembro del crew está especializado en una función distinta del ciclo de entrega.

ORQUESTACIÓN

YUI

DISEÑO Y PLAN

TAKU • ARQUITECTURA

KEI • PLANIFICACIÓN

CONSTRUCCIÓN

SHIN • TESTING

KOU • IMPLEMENTACIÓN

CALIDAD Y OPERACIÓN

SHO • REVISIÓN

RIN • CI/OPS

EI • SEGURIDAD

CONOCIMIENTO

CHI • LECCIONES

FUMI • DOCUMENTACIÓN

SUI • DEBUGGING

El crew mismo es stateless entre invocaciones. Coordina exactamente con lo que acaban de ver: plans, specs y knowledge files, versionados en filesystem y git. No es una analogía de la charla. Es la charla, corriendo en producción.

Gracias. Conectemos.



Conectemos

shoootyou.dev/connect



¿Quieren ver las slides? Este QR

shoootyou.dev/talks/devopsdays-lima-2026

#DODLIMA

#DEVOPSDAYSLIMA



DEVOPSDAYS.PE

27-28 AGO 2026 · LIMA CONVENTION CENTER